

JAVA

09

Microservices with Spring Boot & Spring Cloud

Break the monolith — build systems that scale independently, deploy faster, and survive real-world traffic.

DURATION

6 Sessions

LEVEL

Advanced

MODULE

Module 9

UPDATED

June 2026

IN THIS MODULE

01

Microservices Concepts & Architecture

02

Creating Microservices with Spring Boot

03

Service Discovery using Eureka

04

API Gateway & Load Balancing

05

Inter-Service Communication (Feign Client)

06

Monitoring & Logging (Actuator + Prometheus)

07

Mini Project 12 — Campus Connect Microservices Edition

SESSION

01

60 MIN · ARCHITECTURE

Microservices Concepts & Architecture

◆ Learning Objectives

- Explain the difference between a monolithic and a microservices architecture
- Identify the key advantages of microservices: independent deployment, scaling, and fault isolation
- Decompose the Campus Connect Portal into distinct microservices with clear responsibilities
- Sketch a high-level microservices architecture diagram showing services, communication, and databases
- Recognise the trade-offs and real complexity that come with adopting microservices

01 The Monolith Problem

Most applications start as a monolith — one codebase, one database, one deployable unit. When Flipkart launched in 2007, everything ran as a monolith. By 2013, during Big Billion Day, a bug in the payment module could crash search, cart, and checkout simultaneously — because they all shared the same running process. One deploy to fix one thing meant risking everything.

Monolithic Architecture

Single codebase and deployment. All modules — auth, payments, catalog — are tightly coupled. One bug brings everything down. Scaling means replicating the entire app even if only one feature needs more capacity.

Microservices Architecture

Each capability is an independent, deployable service. Services communicate via REST APIs. One service failure stays contained. Each service scales, deploys, and evolves at its own pace independently.

CONCEPT — What is a Microservice?

A microservice is a small, independently deployable unit that does ONE business capability well. In Swiggy: OrderService handles only orders, RestaurantService handles only restaurant data, DeliveryService tracks only riders. Each runs on its own server and communicates with others over REST — just like your React frontend calls a Spring Boot backend API.

02 Why Microservices? Real Advantages

Zomato serves 10+ crore users across 800+ cities in India. During lunch hours, the restaurant-listing service gets 10x more traffic than the payment service. With microservices, Zomato scales only the listing service — spinning up 20 extra instances just for that feature — without touching or restarting payment or reviews. This saves both infrastructure cost and avoids unnecessary deployment risk.

Advantage

What It Means

Indian Real-World Example

Advantage	What It Means	Indian Real-World Example
Independent Deployment	Ship one service without releasing others	Paytm updates KYC flow without wallet downtime
Independent Scaling	Scale only the service under load	IRCTC scales booking engine during Tatkal window
Technology Freedom	Each service can use a different tech stack	Zerodha uses Python for analytics, Java for order engine
Fault Isolation	One service failure stays contained	Swiggy rider-tracking crashes; ordering still works normally
Team Autonomy	Small teams own and ship individual services end-to-end	Amazon two-pizza team model adopted at Flipkart and Meesho

ENTERPRISES USING MICROSERVICES GLOBALLY

86%

Source: O'Reilly Microservices Adoption in Enterprise Survey, 2023

03 Decomposing Campus Connect into Microservices

The Campus Connect Portal from earlier modules is a monolith — one Spring Boot app doing everything. Now we apply the architect mindset: every distinct business capability becomes its own service with its own codebase, its own database, and its own port. This is the Single Responsibility Principle applied at the service level.

Service Name	Single Responsibility	Port
UserService	Registration, login, and profile management	8081
EventService	Create, read, update, and delete campus events	8082
NotificationService	Send confirmation emails and user alerts	8083
API Gateway	Single entry point — routes all client requests	8080

TRY THIS LIVE

Open draw.io (free at app.diagrams.net) or a blank sheet. Draw four boxes: UserService, EventService, NotificationService, API Gateway. Draw arrows showing which service talks to which. Attach a separate database cylinder to UserService and EventService. This is your Campus Connect microservices architecture diagram — you will build the real running version across Sessions 2 through 6.

04 Challenges and When NOT to Use Microservices

Fetching 'all events registered by User ID 5' now requires a network call from EventService to UserService. What if EventService is down? What if the network adds 300ms latency? What if both services update a shared record at the same time? These distributed-system problems — partial failure, network latency, and data consistency — are the real cost of microservices. We address each of them with Eureka, Feign Client, and Resilience4J across the next four sessions.

PRO TIP — Monolith First, Microservices Later

Martin Fowler, who popularised the term, strongly advises: do not start a greenfield project as microservices. Build a well-structured monolith first. Identify which modules need independent scaling under real production load — then extract those specific parts. Most Ahmedabad startups that jump to microservices on day one spend 80% of engineering time on infrastructure and only 20% building product features.

CASE STUDY — Swiggy's Migration from Monolith to Microservices

Context — Swiggy, headquartered in Bengaluru, is India's largest food delivery platform active in 500+ cities with crores of daily transactions.

Challenge — By 2017, Swiggy's growing monolith caused deployment bottlenecks. Fixing a bug in rider-tracking required redeploying the entire platform, causing regular outages affecting ordering, payments, and restaurant listings simultaneously.

What they did — Swiggy split the monolith into dedicated services — ordering, payments, restaurant discovery, and delivery tracking — each with its own database, code repository, and deployment pipeline. Teams were reorganised around individual service ownership.

Result — Deployment frequency jumped from weekly releases to multiple daily deployments per service. Teams gained full autonomy to ship features without cross-team coordination, dramatically cutting time-to-market.

Takeaway — Service independence means: deploy fast, fail small, and scale only what needs scaling.

Source: Swiggy Engineering Blog, 2019 — engineering.swiggy.com

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Cloud Computing — Week 1: Introduction to Cloud Computing and Distributed Systems (IIT Kharagpur, Prof. Soumya K Ghosh)

Link — <https://nptel.ac.in/courses/106105181>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You can now explain what microservices are, articulate their real advantages, and decompose any application into independent services with clear boundaries.

Explore deeper — Martin Fowler's definitive article on Microservices:
martinfowler.com/articles/microservices.html

Try on your own — List the microservices you would design for a ride-hailing app like Ola — aim for 6 to 8 services, each with one clear responsibility and no overlapping concerns.

Connect the dots — Next session you build UserService and EventService as fully independent, running Spring Boot applications on separate ports with their own dedicated databases.

Going further — Explore Domain-Driven Design (DDD) — specifically bounded contexts, which give a principled method for deciding exactly where microservice boundaries should be drawn.

SESSION

02

60 MIN · IMPLEMENTATION

Creating Microservices with Spring Boot

◆ Learning Objectives

- Bootstrap a Spring Boot microservice project using Spring Initializr with the correct dependencies
- Configure application.properties to assign a unique service name and port to each service
- Build a UserService with working REST CRUD endpoints running on port 8081
- Build an EventService with working REST CRUD endpoints running on port 8082
- Apply the Database-per-Service pattern to keep each service's data completely isolated

01 Bootstrapping a Microservice with Spring Initializr

Every microservice is a standalone Spring Boot application — its own project, its own pom.xml, its own port. Go to start.spring.io and create a project named 'user-service'. Select Maven, Java 17, Spring Boot 3.2.x. Add dependencies: Spring Web, Spring Data JPA, MySQL Driver, Lombok. Companies like PhonePe and Razorpay maintain dozens of such independent projects, each in its own Git repository with its own CI/CD pipeline.

# application.properties — UserService	Place in src/main/resources/
server.port=8081	UserService listens exclusively on port 8081
spring.application.name=user-service	Service name — used by Eureka for discovery later
spring.datasource.url=jdbc:mysql://localhost:3306/userdb	
Dedicated database for UserService only	
spring.datasource.username=root	MySQL username
spring.datasource.password=your_password	MySQL password
spring.jpa.hibernate.ddl-auto=update	Auto-create or update tables on startup
spring.jpa.show-sql=true	Print executed SQL to console for debugging

CONCEPT — One Service, One Responsibility

UserService owns only user data. It has its own MySQL database (userdb), its own pom.xml, and runs on its own port. It does NOT touch event data at all. This strict boundary is what makes the service independently deployable — change the user schema and only UserService restarts. EventService is completely unaffected.

02 Building UserService — Entity and Controller

UserService manages student profiles for Campus Connect. It exposes REST APIs for creating, reading, updating, and deleting users. Think of how Ola manages driver and rider profiles completely independently from its ride-booking engine — changing user profile logic never requires touching the booking service.

```

@RestController                                     Marks this class as a REST API controller
@RequestMapping("/api/users")                       All endpoints in this controller start with /api/users
public class UserController {
    @PostMapping                                     POST /api/users — create a new student user
    public ResponseEntity<User> create(@RequestBody User user) {
    return ResponseEntity.status(201).body(userService.save(user));
    Returns 201 Created with the persisted user
    }
    @GetMapping("/{id}")                             GET /api/users/{id} — fetch one user by ID
    public User getById(@PathVariable Long id) {
        return userService.findById(id);             Fetches from userdb via Spring Data JPA
    }
    @GetMapping                                       GET /api/users — list all users
    public List<User> getAll() { return userService.findAll(); }
    Returns full user list
    @DeleteMapping("/{id}")                         DELETE /api/users/{id} — remove a user
    public void delete(@PathVariable Long id) { userService.delete(id); }
}

```

TRY THIS LIVE

Run UserService with 'mvn spring-boot:run'. In Postman, send POST to <http://localhost:8081/api/users> with JSON body: { "name": "Riya Patel", "email": "riya@ldce.ac.in", "college": "LDCE Ahmedabad" }. You should get a 201 Created response with the saved user object including an auto-generated ID. Then call GET <http://localhost:8081/api/users> to confirm the list.

03 Building EventService

EventService manages all campus events. It runs on port 8082 with its own database (eventdb). There is no import of any UserService class — the two projects are completely decoupled at the code level. This is service autonomy in practice: the EventService team can release a new feature without ever talking to the UserService team.

```

# application.properties — EventService
server.port=8082                                     Separate config file in event-service project
spring.application.name=event-service                EventService runs on a completely different port
spring.datasource.url=jdbc:mysql://localhost:3306/eventdb
                                                     Unique service name for Eureka registration
    Separate DB — never shares userdb
spring.datasource.username=root                       MySQL username
spring.datasource.password=your_password             MySQL password
spring.jpa.hibernate.ddl-auto=update                 Auto-create event tables on startup

```

Service	Endpoint	HTTP Method	Description
UserService	/api/users	POST	Create a new student user
UserService	/api/users/{id}	GET	Fetch one user by ID
UserService	/api/users	GET	List all users
UserService	/api/users/{id}	DELETE	Delete a user record
EventService	/api/events	POST	Create a new campus event
EventService	/api/events/{id}	GET	Fetch one event by ID
EventService	/api/events	GET	List all events
EventService	/api/events/{id}	PUT	Update an existing event

04 Database Per Service — Why It Matters

The most important architectural rule in microservices is: each service owns its data. UserService writes only to userdb. EventService writes only to eventdb. No service reads another service's database directly — it must call the other service's API. This isolation means you can change EventService's schema without ever touching or restarting UserService. This is how Zerodha's trading engine can be updated without affecting the portfolio reporting service.

PRO TIP — Never Share a Database Between Two Microservices

If two services share the same database table, they are not truly independent — a schema migration in one service can break the other at runtime. This is one of the most common rookie mistakes in microservices. Enforce database-per-service as a hard architectural rule from day one. In production, services can even use different database engines: UserService on MySQL, EventService on PostgreSQL, if the data model suits it.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Cloud Computing — Week 2: Service-Oriented Architecture and Distributed Data Management (IIT Kharagpur, Prof. Soumya K Ghosh)

Link — <https://nptel.ac.in/courses/106105181>

WHERE TO GO NEXT

UserService and EventService now run independently on separate ports with separate databases. The next challenge is making them findable without hardcoded IP addresses.

Explore deeper — Spring Boot REST API official reference:
docs.spring.io/spring-boot/reference/web/servlet.html

Try on your own — Add a third endpoint to EventService: GET /api/events/user/{userId} that returns all events where the createdByUserId matches. This endpoint will be called by Feign Client in Session 5.

Connect the dots — Next session introduces Eureka Server — a dynamic service registry that lets every service find every other service by name instead of hardcoded localhost ports.

Going further — Read about the Saga Pattern — how microservices coordinate transactions that span multiple services and databases without a single shared transaction boundary.

SESSION

03

60 MIN · INFRASTRUCTURE

Service Discovery using Eureka

◆ Learning Objectives

- Explain why dynamic service discovery is essential in a production microservices environment
- Set up a Eureka Server as the service registry for the Campus Connect system
- Register UserService and EventService as Eureka clients with correct configuration
- Verify service registration by reading the Eureka dashboard at localhost:8761
- Use a service name instead of a hardcoded URL to resolve a running service instance

01 The Dynamic Discovery Problem

Right now, UserService calls EventService at 'http://localhost:8082'. This works on a laptop, but on AWS or Azure, EventService might run across 10 server instances with dynamically assigned IP addresses. Which one do you hardcode? None. This is exactly the problem Netflix solved with Eureka — a phone directory where every microservice registers itself on startup and any other service can look it up by name, not by IP.

CONCEPT — What is Eureka?

Eureka is Netflix's open-source service registry, now bundled in Spring Cloud Netflix. Every microservice on startup registers its name, IP, and port with Eureka — like checking into a hotel. When UserService needs to call EventService, it asks Eureka: 'Where is event-service right now?' Eureka returns a live instance address. No hardcoded IPs. Services come and go — Eureka always knows who is running.

02 Setting Up Eureka Server

The Eureka Server is a dedicated Spring Boot application — the switchboard operator for your entire microservices system. Create a new project at start.spring.io named 'eureka-server'. Add only one dependency: Eureka Server from Spring Cloud Discovery. It runs on port 8761 by convention — all Spring Cloud clients know to look there by default.

```
# application.properties – eureka-server
server.port=8761
spring.application.name=eureka-server
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

Place in src/main/resources/
Standard Eureka Server port — all clients expect this
Name for this registry service
Server does not register itself in its own registry
Server does not fetch its own registry list

```
// EurekaServerApplication.java – main class
@SpringBootApplication
@EnableEurekaServer
```

Standard Spring Boot entry point
This single annotation activates the full Eureka registry

```
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
        Start the Eureka registry on port 8761
    }
}
```

TRY THIS LIVE

Run the Eureka Server with 'mvn spring-boot:run'. Open a browser and go to <http://localhost:8761>. You will see the Eureka dashboard — a web UI showing all registered services. It will currently say 'No instances currently registered with Eureka'. Keep this tab open — you will see UserService and EventService appear there within 30 seconds of starting them in the next steps.

03 Registering Services as Eureka Clients

Now turn UserService and EventService into Eureka clients. Add the 'Eureka Discovery Client' dependency to each service's pom.xml and annotate the main class with `@EnableDiscoveryClient`. Then add two configuration properties to each application.properties. On startup, the service will automatically register its name, IP, and port with the Eureka Server — zero extra code required.

```
# Add to UserService application.properties      Eureka client registration config
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
    Points UserService to the Eureka Server location
eureka.instance.prefer-ip-address=true           Register using IP address for reliable lookup

# Add the same two lines to EventService application.properties
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
    EventService also registers with the same Eureka Server
eureka.instance.prefer-ip-address=true           Consistent with UserService registration
```

Eureka Property	What It Controls	Default or Recommended Value
eureka.client.service-url.defaultZone	URL of the Eureka Server to register with	http://localhost:8761/eureka/
eureka.instance.prefer-ip-address	Register by IP rather than hostname	true (recommended)
eureka.instance.lease-renewal-interval-in-seconds	How often service sends a heartbeat to Eureka	30 seconds

Eureka Property	What It Controls	Default or Recommended Value
eureka.instance.lease-expiration-duration-in-seconds	Time before Eureka marks a silent service as dead	90 seconds
eureka.client.registry-fetch-interval-seconds	How often client refreshes its local copy of the registry	30 seconds

04 Fetching Service Instances by Name

Once services are registered, any service can find another by name using Spring's `DiscoveryClient`. Instead of hardcoding 'localhost:8082', `UserService` asks Eureka: 'Give me the current address of event-service.' Eureka returns a live instance URI. This makes the system cloud-ready — IPs can change, pods can restart, but the service name remains constant and reliable.

```
@Autowired
private DiscoveryClient discoveryClient;

List<ServiceInstance> instances =
discoveryClient.getInstances("event-service");
    Lookup by spring.application.name, not by IP
ServiceInstance instance = instances.get(0);
String url = instance.getUri().toString();
```

Inject Spring Cloud `DiscoveryClient`
Gives access to the Eureka registry

Retrieve all live instances of event-service

Pick first available instance

Returns dynamic URI like `http://192.168.x.x:8082`

PRO TIP — Use Service Names, Never Hardcoded URLs

The entire purpose of Eureka is to eliminate hardcoded service addresses. If you still write 'http://localhost:8082' anywhere in your codebase, you have added Eureka as decoration but not actually adopted service discovery. From this session onward: every cross-service call must go through a service-name lookup. In Session 5, Feign Client does this automatically — you simply declare the service name in an annotation.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Cloud Computing — Week 3: Distributed Coordination and Service Management (IIT Kharagpur, Prof. Soumya K Ghosh)

Link — <https://nptel.ac.in/courses/106105181>

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Spring Cloud Eureka expert. Explain in simple Hinglish what Eureka Service Discovery is and why it is needed in microservices. Use an analogy of a Zomato delivery system where multiple restaurants (services) register themselves on an app. Explain what happens when a new restaurant opens or closes. Keep it under 150 words.

WHERE TO GO NEXT

UserService and EventService are now registered in Eureka and can discover each other dynamically — no hardcoded IPs anywhere in the codebase.

Explore deeper — Spring Cloud Netflix Eureka reference docs:
docs.spring.io/spring-cloud-netflix/docs/current/reference/html/

Try on your own — Register NotificationService on port 8083 with Eureka as well. Confirm all three services — user-service, event-service, notification-service — appear on the Eureka dashboard at localhost:8761.

Connect the dots — Next session, the API Gateway becomes the single entry point for all client requests — routing them to the correct service through Eureka-backed load balancing.

Going further — Research Eureka High Availability: how do you run two Eureka Server instances in a peer-to-peer setup so the registry itself does not become a single point of failure?

SESSION

04

60 MIN · INFRASTRUCTURE

API Gateway & Load Balancing

◆ Learning Objectives

- Explain why an API Gateway is required as a single entry point in microservices
- Set up a Spring Cloud Gateway project that routes requests to UserService and EventService
- Configure route predicates using application.properties to match URL path patterns
- Enable Eureka-backed load balancing using the lb:// URI scheme in gateway routes
- Apply cross-cutting concerns — request logging and header manipulation — via gateway filters

01 Why Do We Need an API Gateway?

Without a gateway, the React frontend for Campus Connect must know every service URL: `http://localhost:8081` for users, `http://localhost:8082` for events, `http://localhost:8083` for notifications. If any service moves or changes port, the frontend breaks. In production systems like Razorpay or MakeMyTrip, mobile apps and browser clients cannot track dozens of shifting service addresses. The API Gateway gives every client a single stable URL to call.

Without API Gateway

Frontend must know every service URL. Changing one service address breaks all clients. No central place for authentication, logging, or rate limiting. Microservices are directly exposed to the internet.

With API Gateway

Clients call one URL — port 8080. Gateway routes internally to the correct service. Authentication, logging, and rate limiting are handled once in one place. All microservices are hidden and protected behind the gateway.

CONCEPT — Spring Cloud Gateway

Spring Cloud Gateway is a non-blocking, reactive API gateway built on Spring WebFlux. Every incoming request at port 8080 is checked against routing rules called predicates. If a request path matches, the gateway forwards it to the right service — found via Eureka — after optionally passing it through filters. Think of it as a traffic officer at an Ahmedabad highway toll plaza: every vehicle enters through one gate and is directed to the correct lane.

02 Setting Up Spring Cloud Gateway

Create a new Spring Boot project named 'api-gateway' at start.spring.io. Add dependencies: Spring Cloud Gateway, Eureka Discovery Client. Set it to run on port 8080 — the one public-facing port for the entire Campus Connect system. Most gateway behaviour is configured through properties, not Java code.

```
# application.properties — api-gateway
server.port=8080
spring.application.name=api-gateway
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

Gateway also registers with Eureka
spring.cloud.gateway.routes[0].id=user-route
spring.cloud.gateway.routes[0].uri=lb://user-service

lb:// prefix = load-balanced Eureka lookup
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/users/**

Match any request whose path starts with /api/users/
spring.cloud.gateway.routes[1].id=event-route
spring.cloud.gateway.routes[1].uri=lb://event-service

Resolve event-service via Eureka
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/events/**

Match any request whose path starts with /api/events/
```

TRY THIS LIVE

Start all four components in this order: Eureka Server, UserService, EventService, API Gateway. Wait 30 seconds for all registrations. In Postman send GET `http://localhost:8080/api/users` — you should receive the user list even though UserService runs on port 8081. The gateway routed it transparently. Then send GET `http://localhost:8080/api/events` — should return events from EventService on port 8082. You never called those ports directly.

03 Load Balancing with lb:// and Eureka

The 'lb://' URI prefix tells Spring Cloud Gateway to use client-side load balancing backed by Eureka. Start two instances of EventService — one on port 8082 and another on port 8092. Both register with Eureka under the same name 'event-service'. The gateway now distributes requests across both instances in round-robin order. This is exactly how Swiggy handles peak Diwali traffic: spin up more service instances and the gateway balances across all of them automatically.

Gateway Feature	What It Does	Configuration Example
Path Predicate	Match requests by URL path pattern	Path=/api/users/**
Method Predicate	Match by HTTP method	Method=GET,POST
Header Predicate	Match by a specific request header	Header=X-App-Version, 2.0
AddRequestHeader Filter	Inject a header before forwarding	AddRequestHeader=X-Source, CampusConnect

Gateway Feature	What It Does	Configuration Example
StripPrefix Filter	Remove leading path segments before forwarding	StripPrefix=1 removes /api prefix
Retry Filter	Retry failed upstream calls automatically	Retry=3 attempts before returning an error

04 Cross-Cutting Concerns via Filters

One of the biggest benefits of centralising traffic through a gateway is handling cross-cutting concerns once for all services. Authentication (JWT token validation), request and response logging, CORS headers, and rate limiting are all implemented once in the gateway. Every microservice behind it focuses entirely on its own business logic. This mirrors how PhonePe enforces authentication for all its payment APIs through a single gateway layer.

PRO TIP — Use the Gateway for Rate Limiting, Not Individual Services

Spring Cloud Gateway supports rate limiting via the RequestRateLimiter filter backed by Redis. Configure it to allow, say, 100 requests per second per user IP. This protects all downstream services from sudden traffic spikes — like IRCTC during Tatkal booking windows or a product launch on Meesho. Implementing rate limiting in individual microservices duplicates logic and wastes effort; the gateway is the right single enforcement point.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Cloud Computing — Week 4: Load Balancing and Fault Tolerance in Distributed Systems (IIT Kharagpur, Prof. Soumya K Ghosh)

Link — <https://nptel.ac.in/courses/106105181>

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Spring Cloud Gateway expert. In simple Hinglish, explain how an API Gateway works using an example of a student using the Campus Connect app to register for a tech event. Show the full request journey: mobile app calls port 8080, gateway routes to EventService on port 8082, EventService saves data and returns a response. Explain what happens if EventService is busy with 2 running instances. Keep it under 150 words.

WHERE TO GO NEXT

The API Gateway is live — all Campus Connect traffic now flows through a single entry point at port 8080, automatically routed and load-balanced to the correct service.

Explore deeper — Spring Cloud Gateway predicate and filter reference:
docs.spring.io/spring-cloud-gateway/docs/current/reference/html/

Try on your own — Add a third route in the gateway for NotificationService on port 8083. Add a global filter that logs every incoming request method and path to the console using a GlobalFilter bean.

Connect the dots — Next session, Feign Client replaces the verbose DiscoveryClient pattern — making cross-service calls as clean as calling a local Java method with a single annotated interface.

Going further — Explore Spring Security integration with Spring Cloud Gateway for JWT-based authentication that protects all microservices through a single security layer.

SESSION

05

60 MIN · INTEGRATION

Inter-Service Communication with Feign Client

◆ Learning Objectives

- Explain the limitations of using RestTemplate for service-to-service communication
- Add OpenFeign to a Spring Boot microservice and enable it with `@EnableFeignClients`
- Define a Feign client interface that maps to EventService REST endpoints
- Implement a fallback class for graceful degradation when a downstream service is unavailable
- Describe how the Circuit Breaker pattern with Resilience4J prevents cascading failures

01 The Inter-Service Communication Challenge

UserService needs to call EventService to get the list of events a specific user has registered for. Using Spring's RestTemplate, you manually construct the URL string, execute the HTTP call, parse the JSON response, and handle errors. This works — but it is verbose and error-prone. In a system like MakeMyTrip, where 50+ services call each other thousands of times per second, RestTemplate boilerplate becomes a serious maintenance burden across every service.

RestTemplate (Old Approach)

Manually construct URL strings with service names or IPs. Handle HTTP response codes and JSON parsing yourself. No built-in load balancing or retry logic. 8 to 10 lines of plumbing code for a simple GET call. No compile-time contract between the caller and the called service.

Feign Client (Modern Approach)

Declare a Java interface with Spring MVC annotations. Spring generates the HTTP call at runtime automatically. Integrates with Eureka for load balancing out of the box. 3 to 5 lines of interface code replaces 10 lines of RestTemplate. Compile-time type safety between caller and called service.

CONCEPT — OpenFeign

OpenFeign is a declarative HTTP client from Netflix, integrated into Spring Cloud. You write a Java interface that mirrors the REST API of the service you want to call. Spring Boot generates the real HTTP request code at application startup — you never write it. You call the remote EventService as if it were a local Java method. Eureka integration means the actual IP is resolved at runtime, transparently.

02 Setting Up Feign Client in UserService

Add the 'spring-cloud-starter-openfeign' dependency to UserService's pom.xml. Annotate the main application class with `@EnableFeignClients` — this tells Spring Boot to scan for Feign interface definitions and register them as injectable beans. That is the only setup required. Then define the client interface.

```
// Step 1 – Enable Feign in main application class
@SpringBootApplication           Standard Spring Boot entry point
@EnableFeignClients             Activates Feign client scanning across this service
public class UserServiceApplication { ... }

// Step 2 – Declare the Feign client interface
@FeignClient(name = "event-service")   Name must match spring.application.name in
                                       EventService

public interface EventServiceClient {   No @Service annotation — Feign creates the bean
    @GetMapping("/api/events/user/{userId}")   Maps to the endpoint exposed by EventService
    List<EventDTO> getEventsByUser(@PathVariable Long userId);
    Return type auto-deserialized from JSON response
}                                       No implementation class needed — Feign handles it
```

TRY THIS LIVE

Inject EventServiceClient into a UserController method. Add a new endpoint: GET /api/users/{id}/events that calls eventServiceClient.getEventsByUser(id) and returns the result. Start Eureka Server, EventService, UserService, and Gateway in order. Send GET http://localhost:8080/api/users/1/events — you should receive the event list from EventService, retrieved from inside UserService using just one method call. No URL, no parsing, no boilerplate.

03 Handling Failures — Fallback when a Service is Down

What happens if EventService is down when UserService calls it? Without a fallback, Feign throws an exception and the entire user profile request fails — even though user data is perfectly available. This is a cascading failure: one broken service drags down an unrelated service. Zomato handles this by still showing restaurant listings even when the review service is temporarily down — the fallback returns cached or empty data instead of crashing.

```
// Register the fallback on the FeignClient annotation
@FeignClient(name = "event-service", fallback = EventServiceFallback.class)
    Fallback class activates when EventService is unreachable
public interface EventServiceClient {
    @GetMapping("/api/events/user/{userId}")
    List<EventDTO> getEventsByUser(@PathVariable Long userId);
}

// Fallback implementation – must implement the Feign interface
@Component           Must be a Spring-managed bean
public class EventServiceFallback implements EventServiceClient {
    public List<EventDTO> getEventsByUser(Long userId) {
        Called automatically when EventService is down
    }
}
```

```
return Collections.emptyList();
}
}
```

Return safe empty list — user profile request still succeeds

Failure Scenario	Default Feign Behaviour	With Fallback Implemented
Target service is down	FeignException thrown — caller request fails entirely	Fallback returns safe default value
Target service times out	Timeout exception after default 10 seconds	Fallback triggers at timeout threshold
Target returns HTTP 500	FeignException propagates with 500 status details	Fallback can log error and return default
Network unreachable	UnknownHostException propagates up the call chain	Fallback isolates the caller from the failure

04 Circuit Breaker Pattern with Resilience4J

A fallback stops the error from propagating, but it does not stop UserService from hammering a failing EventService with thousands of retried requests per second, making recovery harder. A circuit breaker watches the failure rate: if more than 50% of calls fail in a rolling time window, the circuit opens — all further calls go directly to the fallback for 30 seconds without ever hitting EventService. This gives the failing service time to recover. Add resilience4j-spring-boot3 to your pom.xml and set feign.circuitbreaker.enabled=true in application.properties.

PRO TIP — Never Make a Feign Call Without a Fallback

In any production microservices system — whether it is Flipkart, IRCTC, or a campus project — every Feign call must have a fallback implementation. The question is never 'will downstream services fail?' but 'when they fail, how does the system behave?' A fallback that returns a cached result, an empty list, or a friendly error message is always better than an unhandled exception cascading through 6 services and crashing the entire user experience.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Cloud Computing — Week 5: Fault Tolerance, Resilience, and Service Reliability (IIT Kharagpur, Prof. Soumya K Ghosh)

Link — <https://nptel.ac.in/courses/106105181>

WHERE TO GO NEXT

Services can now call each other by name via Feign with automatic load balancing and fallback protection when any downstream service is unavailable.

Explore deeper — Resilience4J circuit breaker configuration guide:
resilience4j.readme.io/docs/circuitbreaker

Try on your own — Stop EventService manually while UserService is running. Call GET `http://localhost:8080/api/users/1/events` and confirm that the fallback returns an empty list in under 1 second — no error, no crash.

Connect the dots — Next session adds observability to the whole system — Spring Boot Actuator and Prometheus reveal which services are healthy, where latency is building, and what error rates look like in real time.

Going further — Research the Bulkhead pattern from Resilience4J: how to limit the number of concurrent Feign calls to any given service so that one slow downstream cannot exhaust all available thread-pool resources.

SESSION

06

60 MIN · OPERATIONS

Monitoring & Logging with Actuator and Prometheus

◆ Learning Objectives

- Explain why observability — health, metrics, and tracing — is essential in a distributed microservices system
- Add Spring Boot Actuator to all microservices and configure which endpoints are exposed
- Enable Micrometer's Prometheus integration to export metrics in Prometheus-compatible format
- Configure Prometheus to scrape metrics from all Campus Connect services on a schedule
- Connect Grafana to Prometheus and import a Spring Boot dashboard to visualise service health

01 Why Monitoring is Non-Negotiable in Microservices

In a monolith, you look at one log file when something breaks. In a 6-service microservices system, a slow frontend response could be caused by a bottleneck in UserService, a timeout in EventService, a memory leak in NotificationService, or a misconfigured route in the API Gateway. Without centralised monitoring, production debugging is like searching for a short circuit in a building without an electrical diagram. IRCTC's infamous Tatkal crashes taught every Indian backend team this lesson the hard way.

AVERAGE COST OF 1 HOUR OF DOWNTIME

Rs 3.5Cr

Source: Gartner IT Downtime Cost Report, 2023 — estimated for a mid-size Indian e-commerce platform

CONCEPT — Spring Boot Actuator

Spring Boot Actuator is a built-in library that automatically exposes operational HTTP endpoints for any Spring Boot service. Add one Maven dependency and you instantly get: `/actuator/health` (is the service alive?), `/actuator/metrics` (CPU, memory, request counts), `/actuator/info` (build and version details), and `/actuator/prometheus` (metrics in Prometheus format). It is a live doctor's dashboard for your running service — zero extra code required.

02 Setting Up Spring Boot Actuator

Add 'spring-boot-starter-actuator' and 'micrometer-registry-prometheus' dependencies to all microservices — UserService, EventService, and API Gateway. By default only `/actuator/health` and `/actuator/info` are exposed over HTTP. Add the following properties to expose all endpoints and enable Prometheus metrics export. In a real production environment, always protect Actuator endpoints with Spring Security.

Add to application.properties of every microservice

Actuator and Prometheus configuration

`management.endpoints.web.exposure.include=*` Expose all Actuator endpoints over HTTP

`management.endpoint.health.show-details=always`

Show granular health info: DB status, disk space

`management.metrics.export.prometheus.enabled=true`

Enable Prometheus metrics registry

`management.endpoint.prometheus.enabled=true` Activates /actuator/prometheus scrape endpoint

`management.metrics.tags.application=${spring.application.name}`

Tag all metrics with the service name for Grafana filtering

TRY THIS LIVE

With UserService running, open the browser and visit these three URLs in order: (1)

<http://localhost:8081/actuator/health> — should return {status:UP, db:{status:UP}}. (2)

<http://localhost:8081/actuator/metrics> — lists all metric names available. (3)

<http://localhost:8081/actuator/metrics/http.server.requests> — shows total HTTP request count, max response time, and error breakdown. Repeat the same three URLs for EventService on port 8082.

Actuator Endpoint	What It Shows	Primary Use Case
/actuator/health	Service UP or DOWN, plus DB and disk connectivity	Load balancer and Kubernetes health checks
/actuator/metrics	JVM memory, CPU usage, HTTP request counts and error rates	Performance monitoring dashboards
/actuator/prometheus	All metrics in Prometheus text exposition format	Prometheus scraping every 15 seconds
/actuator/env	All environment variables and resolved config values	Config debugging in staging and production
/actuator/loggers	Current log levels — changeable at runtime without restart	Live log-level debugging in production
/actuator/info	App version, build time, Git commit hash	Deployment and release verification

03 Prometheus: Collecting Metrics from All Services

Prometheus is an open-source time-series monitoring system. Every 15 seconds it pulls metrics from all configured services — a process called scraping. It calls /actuator/prometheus on UserService, EventService, and the API Gateway, and stores every number with a timestamp. This builds a historical record: what was the average request

time for EventService at 2:00 PM yesterday? How did JVM heap grow over the last 6 hours? Download Prometheus from prometheus.io and run it with the configuration below.

```
# prometheus.yml — Prometheus scrape configuration file
global:
  scrape_interval: 15s                                Collect metrics from all targets every 15 seconds
scrape_configs:
  Define which services to monitor
  - job_name: user-service                             Label for all metrics from UserService
    static_configs:
      - targets: ['localhost:8081']                   Prometheus calls /actuator/prometheus on port 8081
  - job_name: event-service                             Label for all metrics from EventService
    static_configs:
      - targets: ['localhost:8082']                   Prometheus scrapes EventService metrics
  - job_name: api-gateway                               Monitor the gateway entry point too
    static_configs:
      - targets: ['localhost:8080']                   Scrape API Gateway metrics
```

04 Grafana: Turning Raw Metrics into Dashboards

Prometheus collects raw numbers — Grafana turns them into visual dashboards. Download Grafana from grafana.com, add Prometheus as a data source (URL: <http://localhost:9090>), and import the pre-built Spring Boot dashboard using Grafana Dashboard ID 12900. You instantly get: live request rate per service, error rate over time, JVM heap usage trends, and p95 response time. This is the exact monitoring stack used by Zerodha, Freshworks, and most modern Indian product companies.

PRO TIP — Configure Alerts Before You Go to Production

In Grafana, set alert rules before the application goes live: if error rate exceeds 5% for 2 minutes, or heap memory exceeds 80%, send an alert to a Slack channel or email. Set this up in staging — not after something breaks in production. Engineering teams at CRED and Razorpay run automated Grafana alerts that page the on-call engineer within 60 seconds of a production incident starting.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Cloud Computing — Week 6: Monitoring, Logging, and DevOps Practices in Cloud Environments (IIT Kharagpur, Prof. Soumya K Ghosh)

Link — <https://nptel.ac.in/courses/106105181>

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a DevOps engineer at a Bangalore startup. In simple Hinglish, explain to a junior developer what Spring Boot Actuator and Prometheus do, using the analogy of a car dashboard (speedometer, fuel gauge, engine temperature). Explain what happens when an alert fires because EventService memory is 90% full. Keep it under 150 words.

WHERE TO GO NEXT

All Campus Connect microservices are now observable — health status, request metrics, and error rates are being collected by Prometheus and visualised in Grafana.

Explore deeper — Grafana pre-built Spring Boot dashboard:
grafana.com/grafana/dashboards/12900-spring-boot-statistics

Try on your own — Run a load test on your services using Apache JMeter or Postman Collection Runner with 50 concurrent users. Watch request rate and JVM heap graphs rise and fall in real time in the Grafana dashboard.

Connect the dots — The final session brings everything together: Mini Project 12 integrates all six sessions into one complete, end-to-end Campus Connect Microservices system.

Going further — Explore distributed tracing with Micrometer Tracing and Zipkin — follow a single user request as it travels across the API Gateway, UserService, EventService, and NotificationService, with timing for each hop.

SESSION

07

180 MIN · PROJECT

Mini Project 12 — Campus Connect Microservices Edition

◆ Learning Objectives

- Build a complete microservices system with five independently running Spring Boot services
- Wire Feign Client in UserService to call EventService through Eureka-backed load balancing
- Demonstrate all client traffic flowing through the API Gateway on port 8080 only
- Verify all services are UP using Spring Boot Actuator health endpoints
- Execute an end-to-end Postman test suite validating the full integrated system

01 Project Overview — What You Will Build

This project assembles every concept from Module 9 into one complete, running system. You will split the Campus Connect Portal into independently deployable microservices, connect them through Eureka, route all traffic through an API Gateway, enable inter-service communication via Feign Client, and observe the system with Spring Boot Actuator. This is a production-grade architecture — the same pattern that powers Razorpay, Urban Company, and Nykaa backends.

Service	Single Responsibility	Port	Database
eureka-server	Service registry — all services register here on startup	8761	None
api-gateway	Single entry point — routes all incoming client requests	8080	None
user-service	User registration, login, and profile management	8081	userdb (MySQL)
event-service	Event creation, listing, update, and deletion	8082	eventdb (MySQL)
notification-service	Email alerts for event registrations (optional extension)	8083	None

CONCEPT — The Complete Architecture

React Frontend -> API Gateway (8080) -> [user-service (8081) | event-service (8082)] resolved via Eureka. UserService calls EventService using Feign Client with lb://event-service. All services expose /actuator/health and /actuator/prometheus. Prometheus scrapes all three services every 15 seconds. Grafana visualises the live dashboard. This is a six-layer, production-ready microservices architecture built entirely in Spring Boot.

02 Phase 1 — UserService and EventService

Start with the two core services. Use Spring Initializr to create both projects separately. UserService connects to userdb and exposes CRUD APIs for users plus the `/api/events/user/{userId}` Feign-callable endpoint. EventService connects to eventdb and exposes CRUD APIs for events. Confirm `spring.application.name` is set correctly in each — this name is how Eureka and Feign identify every service.

Recommended folder structure — all projects under one root

campus-connect-microservices/

Root workspace folder — open in IntelliJ as a multi-module project

+-- eureka-server/

Spring Boot project — dependency: Eureka Server

+-- api-gateway/

Spring Boot project — dependencies: Gateway, Eureka Client

+-- user-service/

Spring Boot project — dependencies: Web, JPA, MySQL, Feign, Actuator

+-- event-service/

Spring Boot project — dependencies: Web, JPA, MySQL, Actuator

+-- notification-service/

Optional extension — Spring Boot project with basic REST endpoint

TRY THIS LIVE — Phase 1 Checkpoint

Run UserService on port 8081 and EventService on port 8082 directly (no Eureka yet). Open Postman and verify: (1) POST `http://localhost:8081/api/users` creates a user with 201 Created. (2) POST `http://localhost:8082/api/events` creates an event with 201 Created. (3) GET `http://localhost:8082/api/events/user/1` returns an event list. Both services must work independently before you proceed to Phase 2. Fix any issues here first.

03 Phase 2 — Eureka Server and API Gateway

Create and run the Eureka Server project first on port 8761. Add the Eureka Client dependency to UserService and EventService, configure their `application.properties` to register with Eureka, and restart them. Confirm both appear on the Eureka dashboard at `localhost:8761`. Then create the API Gateway project and add routes for both services using the `lb://` prefix for Eureka-backed load balancing.

api-gateway application.properties — final routing config

server.port=8080

Single public-facing port for the entire system

spring.application.name=api-gateway

Gateway registers in Eureka too

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

Points gateway to Eureka registry

spring.cloud.gateway.routes[0].id=user-route

Route definition for UserService

```
spring.cloud.gateway.routes[0].uri=lb://user-service
    lb:// = Eureka-backed load balancing
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/users/**
    Match all /api/users/ requests
spring.cloud.gateway.routes[1].id=event-route    Route definition for EventService
spring.cloud.gateway.routes[1].uri=lb://event-service
    Resolve event-service via Eureka
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/events/**
    Match all /api/events/ requests
```

PRO TIP — Always Follow This Startup Order

Start services in this exact sequence every time: (1) Eureka Server — wait 15 seconds. (2) UserService and EventService — wait 30 seconds for both to fully register with Eureka. (3) API Gateway last — it needs Eureka populated before it can resolve lb:// routes. Use /actuator/health on each service to confirm it is UP before starting the next. This order matters — skipping it causes intermittent 'Service Unavailable' errors that are confusing to debug.

04

Phase 3 — Feign, Actuator, and Final Testing

Add @EnableFeignClients to UserService's main class and define the EventServiceClient interface with a fallback. Add spring-boot-starter-actuator and micrometer-registry-prometheus to all services and configure management.endpoints.web.exposure.include=* in application.properties. Restart all services in order. Run the complete Postman test suite below — every request must go through port 8080 only. Direct calls to ports 8081 and 8082 are not allowed in final testing.

Test Scenario	Postman Request via Gateway	Expected Result
Create a student user	POST http://localhost:8080/api/users	201 Created — user saved in userdb
Create a campus event	POST http://localhost:8080/api/events	201 Created — event saved in eventdb
List all users	GET http://localhost:8080/api/users	200 OK — full list from userdb
Get events by user via Feign	GET http://localhost:8080/api/users/1/events	200 OK — event list from EventService via Feign
Eureka dashboard check	GET http://localhost:8761 (browser)	All 4 services show as UP instances
Actuator health check	GET http://localhost:8081/actuator/health	{status: UP, db: {status: UP}}

TRY THIS LIVE — Resilience Test

With the full system running: stop EventService manually. Now call GET `http://localhost:8080/api/users/1/events` via Postman. The Feign fallback should return an empty list — no crash, no error propagation. UserService is still alive and serving all other endpoints. Restart EventService. Within 30 seconds, Eureka re-registers it and Feign calls resume normally. This is the fault-isolation guarantee of a well-built microservices system.

“Building microservices is not about the technology stack — it is about building a system that stays alive under pressure, scales only what needs to scale, and lets teams ship features without waiting for each other.”

WHERE TO GO NEXT

You have built a complete, production-pattern microservices system from scratch. UserService, EventService, Eureka, Gateway, Feign, Actuator — all integrated and tested end to end.

Explore deeper — Add a Docker Compose file to the root folder to start all 5 services with a single 'docker-compose up' — the standard production workflow at every Indian tech company.

Try on your own — Deploy the Eureka Server and API Gateway to a free cloud tier on Render or Railway. Make them publicly accessible and call your endpoints from a phone connected to mobile data.

Connect the dots — This session completes the Java Full Stack Career Program. You can now architect, build, test, and deploy a full-stack Java microservices application end to end.

Going further — Kubernetes is the next frontier: K8s Pods, Services, and Ingress Controllers replace Eureka, Spring Cloud Gateway, and manual deployment at true production scale. Start with Minikube for local K8s practice.

REVIEW



INTERVIEW PREP · MODULE 9

Interview Questions

◆ **Microservices with Spring Boot, Spring Cloud**

Rehearse each answer out loud and aim for a 60-second response. Use the hint only as a nudge if you are stuck. Compare your attempt with the model answer afterward to find gaps.

1

What is a monolithic application, and how does it differ from a microservices architecture?*Think about how all features are packaged and deployed in each approach.***MODEL ANSWER**

A monolithic application packages all features — user management, event management, notifications, payments — into a single deployable unit (one WAR or JAR). If one feature needs an update, the entire application must be redeployed. In a microservices architecture, each feature is a separate, independently deployable service with its own codebase, database, and deployment pipeline. Services communicate over HTTP (REST) or messaging. Microservices enable teams to deploy, scale, and update each service independently without affecting the others.

2

List two main advantages of using microservices over a monolithic architecture.*Consider deployment and scalability.***MODEL ANSWER**

Independent deployment: you can update and deploy the EventService without touching or restarting the UserService. If a deployment fails, only that service is affected. Independent scalability: if event searches are under heavy load, you can spin up three instances of EventService without scaling the entire application. Additional advantages include technology flexibility (each service can use a different tech stack), fault isolation (a crash in one service does not cascade to all), and smaller, more focused codebases that are easier for individual teams to own.

3

Explain, in your own words, why independent deployment is important in a microservices setup.**MODEL ANSWER**

In a monolith, a single line change requires rebuilding, testing, and redeploying the entire application — risky and slow. Independent deployment means each team owns a small service and can release changes on their own schedule without coordinating with every other team. A bug fix in the notification service can go live in minutes without a company-wide deployment window. This dramatically reduces the risk per release because each change is small and contained. It is also why microservices pair so naturally with CI/CD pipelines.

4

In the context of a campus management system, what could be two separate microservices you might create? Briefly explain their responsibilities.

Think about user-related tasks versus event management.

MODEL ANSWER

UserService: handles registration, login, profile management, and password reset for students and staff. It owns the users table and issues authentication tokens. EventService: handles creation, listing, registration, and cancellation of campus events. It owns the events and event_registrations tables and exposes REST APIs for event operations. These two services communicate when EventService needs to validate that the user registering for an event exists — it calls UserService via REST or Feign Client rather than sharing a database.

5

How would you create a simple REST API in Spring Boot for managing users (CRUD operations)? Briefly outline the main steps.

Mention project setup, controller, and repository.

MODEL ANSWER

Create a Spring Boot project with Spring Web, Spring Data JPA, and MySQL Driver starters. Define a User @Entity with fields and a UserRepository extending JpaRepository. Create a UserService with methods for findAll, findById, save, and deleteById. Build a UserController annotated @RestController with @GetMapping, @PostMapping, @PutMapping, @DeleteMapping methods that delegate to UserService. Configure application.properties with the DB connection. Run and test each endpoint in Postman. This is the complete, standalone UserService that will later become a microservice.

6

What is the purpose of REST API communication between microservices?

Focus on how services interact and exchange data.

MODEL ANSWER

Since each microservice runs as an independent process with its own database, they cannot share in-memory objects or query each other's database directly. REST API calls are the standard way for one service to request data or trigger actions in another. For example, when a student registers for an event, EventService calls UserService's GET /users/{id} to confirm the user exists and is active. This keeps services loosely coupled — EventService only needs to know the URL and contract of UserService's API, not its internal implementation.

7

Write a basic Spring Boot controller method to handle a GET request for fetching all users.

Use @GetMapping in your answer.

MODEL ANSWER

```
'@RestController @RequestMapping("/users") public class UserController { @Autowired private UserService userService; @GetMapping public ResponseEntity<List<User>> getAllUsers() { return ResponseEntity.ok(userService.findAll()); } }'. @GetMapping on the class-level @RequestMapping("/users") maps GET /users to this method. The service queries the repository, and Spring's Jackson serialiser converts the List<User> to a JSON array. ResponseEntity.ok() sets the 200 status automatically.
```

8

What is Eureka in the Spring Cloud ecosystem, and why is it used?

Think about service registration and discovery.

MODEL ANSWER

Eureka is a service registry — a central server where each microservice registers its name and network address (host and port) at startup, and sends periodic heartbeats to confirm it is still alive. When one service needs to call another, it asks Eureka for the registered instances of that service by name instead of hardcoding an IP. This solves the problem of dynamic addresses in cloud environments where service instances start, stop, and scale horizontally — you never need to update configuration files when an instance's address changes.

9

Describe how a microservice registers itself with a Eureka server.

Focus on annotations and configuration.

MODEL ANSWER

Add the `spring-cloud-starter-netflix-eureka-client` dependency. Annotate the main class with `@EnableEurekaClient` (or rely on Spring Boot auto-configuration in newer versions). In `application.properties`, set: `'eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka/'` and `'spring.application.name=user-service'`. On startup, the service sends a registration request to Eureka with its name, host, port, and metadata. Eureka stores this and begins expecting heartbeats every 30 seconds. If heartbeats stop, Eureka removes the instance from the registry after a timeout.

10

How can one microservice discover the address of another microservice using Eureka?

Mention how clients fetch instance information programmatically.

MODEL ANSWER

A Eureka client (using the `spring-cloud-starter-netflix-eureka-client` dependency) automatically synchronises the service registry from the Eureka server. When you use Spring Cloud's `@LoadBalanced RestTemplate` or a Feign Client, you address the target service by its `spring.application.name` (e.g., `'http://user-service/users'`) rather than a hardcoded URL. The load-balancer intercepts the call, looks up `'user-service'` in the local registry cache, picks a healthy instance, and resolves the actual IP and port — transparent to your code.

11

If your `UserService` and `EventService` are both registered with Eureka, what happens when `EventService` goes down? How does Eureka help?

Think about service registry and availability.

MODEL ANSWER

When `EventService` stops sending heartbeats, Eureka marks it as unavailable and removes it from the registry after the eviction timeout (default 90 seconds). Any service that was about to call `EventService` will see no healthy instances in the registry and receive a timeout or error — which can be handled with a fallback. Eureka helps by ensuring callers do not keep sending requests to a dead instance. If multiple `EventService` instances were registered, Eureka automatically routes callers to the remaining healthy ones, maintaining partial availability.

12

What is the purpose of an API Gateway in a microservices architecture?

Consider how it manages requests between clients and services.

MODEL ANSWER

An API Gateway is a single entry point for all client requests. Instead of a React frontend knowing the individual URLs of UserService, EventService, and NotificationService, it only knows the gateway URL. The gateway routes each request to the correct microservice based on the path (e.g., /users/** -> UserService, /events/** -> EventService). It also centralises cross-cutting concerns: authentication (check token once at the gateway), rate limiting, SSL termination, and logging — so each microservice does not have to implement these independently.

13

Explain how Spring Cloud Gateway routes a request to the appropriate microservice.

MODEL ANSWER

Spring Cloud Gateway reads route configuration from application.yml. Each route has a predicate (condition) and a URI. Example: 'predicates: - Path=/events/**, filters: - StripPrefix=1, uri: lb://event-service'. When a request for /events/5 arrives, the Path predicate matches, StripPrefix removes the prefix segment, and 'lb://event-service' tells the load balancer to look up 'event-service' in Eureka and forward the request to a healthy instance. 'lb://' signals load-balanced routing; Ribbon or Spring Cloud LoadBalancer picks the instance.

14

What is load balancing, and why is it important in microservices?

Think about distributing traffic among multiple service instances.

MODEL ANSWER

Load balancing distributes incoming requests across multiple instances of a service to prevent any single instance from becoming overwhelmed. If EventService has three running instances, a load balancer routes each request to a different instance in turn (round-robin) or based on load. In microservices, services scale horizontally (more instances) to handle traffic spikes, so load balancing is essential for utilising all instances evenly. Without it, all traffic hits one instance while others sit idle. Spring Cloud uses client-side load balancing via Ribbon or Spring Cloud LoadBalancer.

15

How does Ribbon work with Eureka to provide client-side load balancing?

Ribbon uses Eureka's service registry to choose an instance.

MODEL ANSWER

Ribbon is a client-side load balancer that runs inside the calling service, not as a separate proxy. When you use a @LoadBalanced RestTemplate to call 'http://event-service/events', Ribbon intercepts the call, queries the local Eureka registry cache for all instances of 'event-service', applies a load-balancing algorithm (round-robin by default) to pick one instance, replaces 'event-service' with the actual host:port, and forwards the request. This means each client independently balances its own calls without a central proxy — reducing infrastructure complexity.

16

Suppose you want all requests to `/events/**` to go to the `EventService`. How would you configure this in Spring Cloud Gateway?

Think about route predicates and target URIs.

MODEL ANSWER

In `application.yml` of the Gateway: `'spring.cloud.gateway.routes: - id: event-route, uri: lb://event-service, predicates: - Path=/events/**, filters: - StripPrefix=0'`. This tells the gateway: match any path starting with `/events/`, look up 'event-service' in Eureka (`lb://` prefix), and forward the request as-is. Set `StripPrefix=1` if you want to remove the `/events` prefix before forwarding. The gateway combines this with Eureka to always route to healthy `EventService` instances automatically.

17

What is a Feign Client, and how does it simplify inter-service communication?

Focus on how it handles REST calls.

MODEL ANSWER

Feign Client is a declarative HTTP client in Spring Cloud. Instead of writing `HttpClient` or `RestTemplate` boilerplate, you define an interface annotated `@FeignClient` and declare methods that mirror the target service's REST endpoints. Spring Cloud generates the implementation at runtime. It integrates with Eureka for service discovery (use the service name as the URL), Ribbon for load balancing, and can be configured with fallbacks for resilience. The result is inter-service calls that look like local method calls, keeping the code clean and readable.

18

Write a simple Feign Client interface for calling the `EventService`'s `/events` endpoint.

Use `@FeignClient` and method mapping.

MODEL ANSWER

```
@FeignClient(name="event-service") public interface EventServiceClient {
    @GetMapping("/events") List<Event> getAllEvents();
    @GetMapping("/events/{id}") Event getEventById(@PathVariable Long id);
}
```

Annotate the main class with `@EnableFeignClients` to activate scanning. Inject `EventServiceClient` with `@Autowired` and call its methods as if they were local. Feign handles the HTTP call to Eureka-discovered instances of 'event-service', deserialises the JSON response into `List<Event>`, and handles errors.

19

How would you handle errors or failures when one microservice calls another using Feign?

Mention fallback methods or error handling strategies.

MODEL ANSWER

Use Resilience4J's Circuit Breaker with Feign: annotate `@FeignClient` with `'fallback=EventServiceFallback.class'`. `EventServiceFallback` implements `EventServiceClient` and returns safe defaults (empty list, null object) when the real service is unavailable. The circuit breaker opens after a threshold of failures, preventing further calls to the broken service for a cooldown period. You can also configure a `FeignErrorDecoder` to map specific HTTP error codes (404, 503) to custom exceptions. Logging the failure helps with debugging distributed call chains.

20

Give an example scenario where `UserService` needs to fetch event details from `EventService`. How would you implement this using Feign?

MODEL ANSWER

Scenario: `UserService` needs to return a user's profile along with all events they are registered for, which are stored in `EventService`. In `UserService`, create `@FeignClient(name="event-service") EventServiceClient` with method `@GetMapping("/events/user/{userId}") List<Event> getEventsByUser(@PathVariable Long userId)`. In `UserService`'s profile endpoint, call `List<Event> events = eventServiceClient.getEventsByUser(userId)` and include the list in the `UserProfileResponse` DTO. Feign handles the HTTP call transparently; Eureka resolves 'event-service' to a live instance.

21

What is the purpose of Spring Boot Actuator in microservices?

Think about application monitoring and health checks.

MODEL ANSWER

Spring Boot Actuator exposes built-in HTTP endpoints that provide operational information about the running application. The `/actuator/health` endpoint returns UP or DOWN status — used by Kubernetes, load balancers, and Eureka to decide whether to route traffic to the instance. The `/actuator/metrics` endpoint exposes JVM memory usage, HTTP request counts, database connection pool stats, and custom business metrics. `/actuator/info` shows build metadata. Adding 'spring-boot-starter-actuator' to `pom.xml` enables all of this with zero code.

22

How can Prometheus and Grafana be used together to monitor your microservices?

One collects metrics, the other visualizes them.

MODEL ANSWER

Prometheus is a time-series database that scrapes metrics from your microservices at regular intervals. Add the Micrometer Prometheus registry to each Spring Boot service; it exposes metrics at `/actuator/prometheus` in Prometheus format. Configure Prometheus to scrape each service's endpoint. Grafana connects to Prometheus as a data source and lets you build real-time dashboards — CPU usage, request rates, error rates, and response time histograms — with visual alerts when thresholds are exceeded. Together they give full observability across all your microservices.

23

Which actuator endpoints would you enable to check the health and metrics of your microservices?

Think about `/actuator/health` and `/actuator/metrics`.

MODEL ANSWER

Enable in `application.properties`:

`'management.endpoints.web.exposure.include=health,info,metrics,prometheus'`. `/actuator/health` shows UP/DOWN and details about database, disk, and downstream service health. `/actuator/metrics` lists all available metric names; `/actuator/metrics/http.server.requests` shows request count and latency.

`/actuator/prometheus` exposes all metrics in a format Prometheus can scrape. Restrict sensitive endpoints (env, beans, shutdown) from public access with security configuration — only expose health for load balancer probes.

24

Describe how you would integrate Prometheus with a Spring Boot microservice for monitoring.

Consider dependencies and configuration.

MODEL ANSWER

Add to pom.xml: 'spring-boot-starter-actuator' and 'micrometer-registry-prometheus'. In application.properties: 'management.endpoints.web.exposure.include=prometheus' and 'management.endpoint.prometheus.enabled=true'. The /actuator/prometheus endpoint now serves metrics. In prometheus.yml, add a scrape job: 'job_name: user-service, static_configs: - targets: ["user-service-host:8080"]'. Prometheus polls this endpoint every 15 seconds. The service name and instance labels allow Grafana dashboards to filter by service and show per-service health trends.

25

In your mini project splitting Campus Connect into User and Event services, what role does Eureka play?

Think about how services find and communicate with each other.

MODEL ANSWER

Eureka acts as the phone book for the system. When UserService starts, it registers itself as 'user-service' with its host and port. When EventService starts, it registers as 'event-service'. When EventService needs to call UserService (to validate a user before allowing event registration), it looks up 'user-service' in the Eureka registry and gets the current address. If UserService restarts on a different port or if more instances are added for load, EventService automatically discovers the change — no configuration files need updating.

26

How would you demonstrate API Gateway routing in your Campus Connect microservices project?

MODEL ANSWER

Run Eureka Server, UserService, EventService, and Spring Cloud Gateway. Configure the gateway to route /users/** to 'lb://user-service' and /events/** to 'lb://event-service'. All Postman requests go to the single gateway URL (e.g., localhost:8080). Show that GET localhost:8080/users returns the user list (routed to UserService) and GET localhost:8080/events returns the event list (routed to EventService). Demonstrate that the frontend only needs one base URL, regardless of how many internal services exist. Show in the gateway logs that routing decisions are being made.

27

If your monitoring dashboard shows that EventService is unhealthy, what steps would you take to investigate and resolve the issue?

Consider logs, actuator endpoints, and service dependencies.

MODEL ANSWER

First, call /actuator/health on the EventService instance directly to see which component is DOWN (database, Eureka connection, disk). Check the application logs for stack traces. If the database health is DOWN, verify the MySQL connection string, credentials, and that the database server is reachable. If it is a Eureka connectivity issue, check the Eureka server is running and the serviceUrl in application.properties is correct. Check resource usage (memory, CPU) via /actuator/metrics. Restart the service if the issue is transient; fix configuration if it is a settings problem.

28

Suppose you need to add a new `NotificationService` to your Campus Connect microservices system. What steps would you follow to register it with Eureka and make it accessible via the API Gateway?

MODEL ANSWER

Create a new Spring Boot project with `eureka-client`, `web`, and `data-jpa` starters. Set `'spring.application.name=notification-service'` and `'eureka.client.serviceUrl.defaultZone'` in `application.properties`. Annotate the main class with `@SpringBootApplication` (Eureka client auto-configures from the dependency). Add a route in the Gateway's `application.yml`: `path /notifications/** -> 'lb://notification-service'`. Start `NotificationService` — it registers with Eureka automatically. The gateway now routes `/notifications/**` to it. Existing services remain untouched.

29

Integrative: How do API Gateway, Eureka, and Feign Client work together to enable seamless communication between `UserService` and `EventService`?

Think about routing, discovery, and REST calls.

MODEL ANSWER

A client request for `/events` arrives at the API Gateway. The gateway looks up `'event-service'` in Eureka and routes the request to a healthy `EventService` instance. Inside `EventService`, processing an event registration requires validating the user. `EventService` uses a Feign Client annotated `'@FeignClient(name="user-service")'` to call `UserService`. Feign asks Eureka for `'user-service'` instances, Ribbon picks one, and Feign makes the HTTP call — all transparently. Each layer has a clear role: Gateway handles external routing, Eureka handles discovery, and Feign handles internal service-to-service calls.

30

Integrative: Explain how monitoring with Actuator and Prometheus can help you quickly identify and resolve issues in a load-balanced microservices environment.

Focus on metrics, health checks, and traffic distribution.

MODEL ANSWER

With Actuator exposing `/actuator/health` and `/actuator/prometheus` on every instance, and Prometheus scraping all of them, Grafana shows per-instance metrics side by side. If one `EventService` instance has a rising error rate while others are healthy, you can identify it quickly in the dashboard and remove it from the load balancer rotation. HTTP request rate metrics show which instances are receiving traffic, confirming load balancing is working. Database connection pool metrics reveal if one instance is exhausting connections. This level of visibility would be impossible in a monolith.

31

Integrative: In your Campus Connect project, how would you ensure that user-related requests are always routed to healthy instances of `UserService`, even if some instances go down?

Consider Eureka's registry and the load balancer's behavior.

MODEL ANSWER

`UserService` instances register with Eureka and send heartbeats every 30 seconds. Eureka marks an instance unhealthy if heartbeats stop for 90 seconds and removes it from the registry. The API Gateway uses `'lb://user-service'` which delegates to Spring Cloud LoadBalancer — it only routes to instances currently in the Eureka registry. Feign Clients calling `UserService` from other services also use the same Eureka-backed load balancer. Add Resilience4J circuit breaker fallbacks so that even during the 90-second eviction window, callers get a graceful fallback rather than hanging on a dead instance.

32

How could you use an AI assistant like ChatGPT or GitHub Copilot to help you troubleshoot a failing microservice or debug a Spring Boot configuration issue?

Mention how you would verify AI suggestions before applying them.

MODEL ANSWER

Paste the full stack trace, the relevant application.properties, and the error description into ChatGPT. It can identify misconfigurations (wrong Eureka URL format, missing @EnableFeignClients), explain why a bean is not found, or suggest the correct Feign fallback syntax. Copilot can autocomplete Feign Client methods based on your target service's controller. Before applying: confirm the suggestion works with your Spring Cloud version (APIs differ between 2020, 2021, and 2022+ versions), test in isolation, check the official Spring Cloud documentation, and run integration tests to confirm the fix does not break other service interactions.

33

If you use an AI tool to generate code for a Feign Client or an Actuator configuration, what would you check before adding it to your project?

Think about correctness, security, and compatibility.

MODEL ANSWER

Verify the @FeignClient name matches the exact spring.application.name of the target service (case-sensitive matters). Confirm the method signatures (HTTP method, path, return type) match the target controller's actual endpoints. Check the Spring Cloud version in pom.xml — Ribbon vs. Spring Cloud LoadBalancer APIs differ and the generated code may use the wrong one. For Actuator config, ensure sensitive endpoints (env, beans) are not exposed publicly. Test the Feign Client against a running instance using Postman first, then run integration tests that spin up both services to confirm the generated interface works end to end.